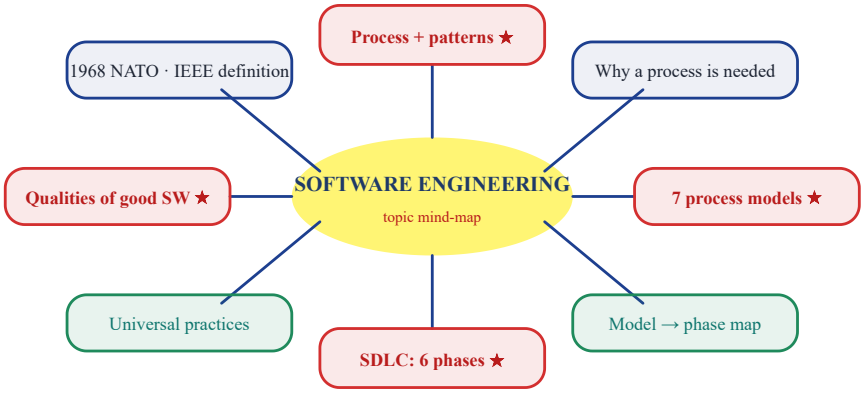
Software Engineering: Concepts, Paradigms & SDLC

What SE is · good-software qualities · process & patterns · 7 models · 6 SDLC phases — in 12 sheets



↑ the whole topic on one page: red = high-yield, green = applied

Syllabus coverage

"Software Engineering and Information System Design — Concepts of **Software Engineering**, **Software Engineering paradigms**, Different phases of **software System Development**, Different types of information, qualities of information." Metrics, risk numbers and scheduling are in the Project Management lesson.

High-yield list

① SE coined: NATO, Garmisch 1968
② IEEE: systematic + quantifiable
③ Spiral = Boehm, risk-driven
④ V-model pairs phase ↔ test level
⑤ SRS = requirements deliverable
⑥ Cost of a fix: ×1 → ×100

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

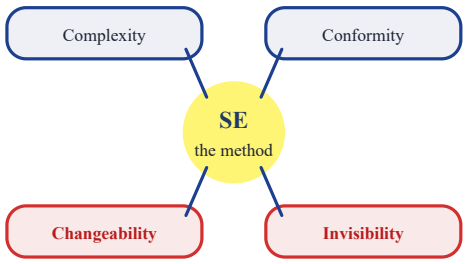
✓ What SE is · why a process …… p. 2
✓ Qualities of good software …… p. 2
✓ Process · framework · practices …… p. 3
✓ 14 patterns · information …… p. 4
✓ Models I–III …… p. 5–7
✓ Comparison of the 7 models …… p. 8
✓ SDLC phases … p. 9 · phase map … p. 10
✓ Night-before p. 11 · Practice p. 12

1 What Is Software Engineering?

Definition: The **systematic, disciplined and quantifiable application of engineering method** to software: how it is specified, designed, built, verified, deployed, maintained and evolved. Coined at the **NATO, Garmisch 1968** conference, which also named the **software crisis**.

Why a process is unavoidable

Attribute of software	Why it forces a process
Complexity	no physical law limits it
Conformity	obeys contracts, standards, law
Changeability	needs move while being built
Invisibility	progress shown only by artefacts



four properties that make software unlike every other engineering artefact

Memory trick: **C-C-C-I** — **C**omplexity, **C**onformity, **C**hangeability, **I**nvisibility. A "which is **not** an attribute?" question is answered with **rigidity** or **fatigue**.

2 Characteristics (Qualities) of Good Software

Quality	Exactly what it means	Keyword
Correctness	gives the output the SRS demands	"meets the spec"
Reliability	no failure under expected conditions	MTBF
Efficiency	uses CPU, memory, I/O optimally	time & space
Maintainability	effort to fix, adapt, enhance	"≈ 70% of cost"
Portability	runs elsewhere with little change	bytecode
Usability	learnable, forgiving interface	user-friendly
Integrity	security: no unauthorised access	confidentiality
Flexibility	new features by local changes	modularity
Testability	verifiable by repeatable tests	verifiability

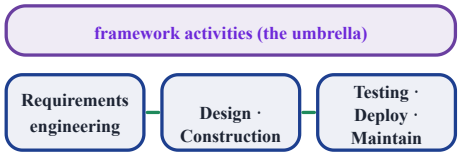
Exam trap: **Correctness cannot be proved in general.** Testing only raises confidence, so "100% correct" is not certifiable.

Exam trap: **Maintainability ≠ usability.** Usability is about the *user*; maintainability is about the *developer*.

3 The Process, Its Framework, Its Universal Practices

Software process: The set of activities, their **order, roles, inputs and outputs**, that turns a requirement into an operating product. A **model / paradigm** is one particular *arrangement* of it — real teams work hybrid.

Term	Means
Process	activities + order + roles + artefacts
Methodology	a process **plus** its rules and paperwork
Model / paradigm	the **shape** of the process
Tool	automates an activity (CASE, IDE)



every activity set wears the umbrella under whatever model is chosen

the generic framework: umbrella over each activity set

The six framework activities — they cut across *every* model

Framework activity	What it does	Artefact
Project planning	estimates effort, cost, schedule; fixes milestones	project plan
Tracking & control	measures progress against the plan, corrects it	status report
Risk analysis	finds technical, schedule, budget, staff risks	risk register
Quality assurance	the QA plan, the standards, the formal reviews	QA plan
Configuration mgmt	baselines, versioning, change control	baselines
Technical reviews	the gate checking each artefact before acceptance	review minutes

Add to the list: **Requirements traceability** (requirement → design → code → test, in an **RTM**), **unit tests written with the code**, **prototyping to explore**, **reuse of libraries and COTS**.

Memory trick: The umbrella spells **P-T-R-Q-C-T**: **P**lan, **T**rack, **R**isk, **Q**uality, **C**onfiguration, **T**eview.

Exam trap: The **generic process framework is not a process model** — it is the umbrella under which any model operates. "Which is the model?" ⇒ waterfall / spiral / V-model, never "framework".

Standards **ISO/IEC 12207** · maturity **CMM / CMMI** 5 levels

4 Generic Process Patterns (14)

What they are: Reusable, named **solutions to recurring process problems**. A lifecycle model is a **selection** of these patterns — none is universally best.

Technology (5) and process (3) patterns

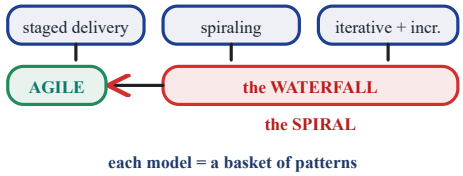
- **Reusable-architecture evolution**
- **Iterative design**
- **Incremental delivery**
- **Evolutionary prototyping**
- **Spiraling**
- **Staged delivery** (process)
- **Discipline of formal methods** (process)
- **Executable specifications** (process)

Practice patterns (6)

- **Tool-based transformation & automation**
- **Computer-aided software engineering (CASE)**
- **Documented rationales for data design**
- **Prototyping for data design**
- **Mathematical formalism in algorithms & data structures**
- **Object-oriented design paradigms**

Exam trap: Patterns are **not** prescriptions. "Which pattern prescribes a disciplined use of mathematics in the design of algorithms and data structures?" → **mathematical formalism**, not "formal methods" — that one is the *process* pattern of staging formal proof.

Models are baskets of patterns: waterfall = staged delivery · spiral = spiraling + risk analysis · agile = iterative + incremental + evolutionary prototyping.



three patterns in, three models out

5 Information: Types and Qualities

Basis	Types & examples
By nature	**Quantitative** (sales, temperature) · **Qualitative** (customer feedback)
By time	**Real-time** (live share price) · **Historical** (last month's sales)
By source	**Internal** (own sales data) · **External** (market research)
By purpose	**Strategic** (expansion) · **Tactical** (resources) · **Operational** (daily stock)

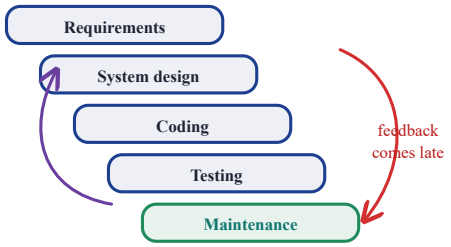
Qualities of good information:

① **Accuracy** — correct, error-free · ② **Relevance** — serves the decision
③ **Completeness** — all needed detail · ④ **Timeliness** — available when needed
⑤ **Reliability** — trustworthy source · ⑥ Understandable
⑦ Consistent · ⑧ Accessible (easy to retrieve)

6 Process Models I — Waterfall, V-Model

Common ground: Every model performs the **same** activities. What differs is the **order**, the **feedback** and the **timing of tests**. Never memorise a model as a list of phases — memorise its **shape**.

6.1 Waterfall (linear / sequential) model



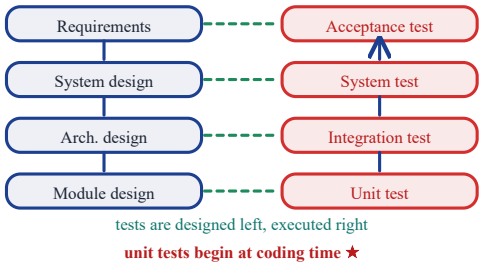
one pass per phase; the loop back is expensive

- **Origin: Benington, IBM, 1956**; **Royce, 1970** drew it and listed its defects.
- **Shape:** phases in strict order, each **signed off** before the next.
- **Fits:** requirements understood and frozen; fixed-price contracts; regulated work.
- **Strengths:** discipline, a document at every gate, easy budgeting, **contract-friendly**.
- **Weaknesses:** no working software until late, wrong requirements found very late, **inflexible to change**.

Exam trap: Royce did **not** propose the waterfall as a recipe for success — his 1970 paper presents it as a model **with serious problems**, to be cured by feeding verification results back into every phase.

6.2 V-Model (verification & validation model)

Left arm — development	Right arm — the matching test level
Requirements	**Acceptance** testing (the user's view)
System design	**System** testing (whole system)
Architectural design	**Integration** testing (modules)
Module design	**Unit** testing (by the developer)

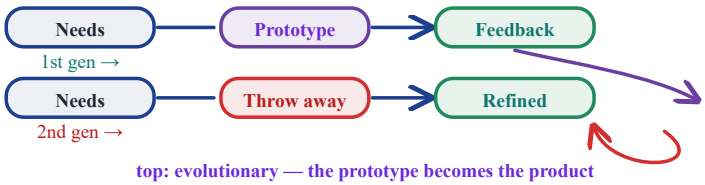


each test level is planned when its design starts

Memory trick: Read the V-model from the **bottom up** for the pairings: **module→unit, architecture→integration, system→system, requirements→acceptance**. "The smaller the design box, the smaller the test."

7 Process Models II — Prototyping, Incremental, Big Bang

7.1 Prototyping model

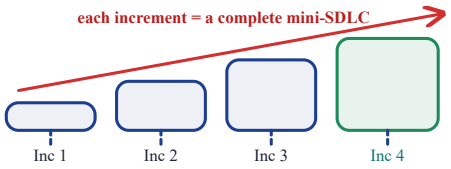


top row = evolutionary (kept and improved); bottom row = throw-away (discarded and rebuilt)

- **Use when requirements are unclear** and must be explored with the user **before** the build.
- **Strength:** kills requirements ambiguity early; the user sees a working thing fast.

- **1st gen / throw-away:** a paper or screen model used only to find requirements, then **discarded**.
- **2nd gen / evolutionary:** improved repeatedly until it **becomes** the product.
- **Weakness:** users may mistake a prototype for the product, so quality shortcuts survive.
- **Trap:** prototyping is **not** "build it twice and keep the worse version".

7.2 Incremental (evolutionary delivery) model



usable slices: the core first, then more

- **Shape:** the system grows in **working, releasable increments**; each runs the whole SDLC. **Increment 1 is the core product**, not a stub.
- **Fits:** requirements can be listed and **prioritised**; the customer can use the system early.
- **Strength:** usable software early, feedback per release, risk spread over time.
- **Weakness:** total cost can exceed waterfall; a later increment re-does **requirements and design**.
- **Trap: iterative ≠ incremental** — iterative repeats a process; incremental adds a slice.

7.3 Big Bang model (build-and-ship, no process)

- **Shape: no phases at all** — requirements, design, coding and testing all at once; one release.
- **Fits:** tiny, low-risk, throwaway experiments where the schedule beats the method.
- **Strength:** zero process overhead, fastest possible first build.
- **Weakness: massive late risk** — no feedback, no intermediate release, unmaintainable afterwards.



everything at once, then one release

Exam trap: Big Bang is **not** "the waterfall done badly" — it is the **absence** of a process.

8 Process Models III — Spiral (Boehm) & RAD

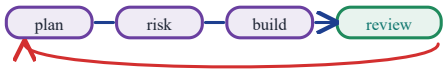


four quadrants, one per turn; later loops cost less

8.1 Spiral model

- **Author: Boehm, 1986**, adapting risk-management practice for large projects.
- **Shape: risk-driven** — each loop does **planning → risk analysis → engineering → evaluation**; the product grows progressively from a prototype.
- **Quadrants:** 1 objectives, alternatives, constraints · 2 evaluate and resolve risks · 3 develop and verify the next-level product · 4 plan the next iteration.
- **Fits:** large, high-risk, expensive-to-fail projects — defence, banking core, safety-critical.
- **Strengths:** makes risk explicit and front-loaded; combines **cyclic** work with **reuse**.
- **Weaknesses:** expensive; needs trained risk analysts; without discipline it is "chaotic waterfall".

> **Memory trick:** Clockwise from the top-left: **O**bjectives → **R**isk → **b**uild → **P**lan (**ORBP**) — one full round per loop.



one loop = plan, risk, build, review — then loop again; the next round is cheaper

8.2 RAD — Rapid Application Development

> **RAD:** A **time-boxed, small-team, component-reuse** approach: analysis, design and prototyping are **fused into one round** and the product is built in about **90 days**. Popularised in the 1980s by **James Martin**.

- **Fits:** small teams, GUI/business applications, requirements understood by both sides.
- **Does not fit:** large mission-critical back-end systems, heavy safety demands, distributed teams.
- **Strength:** extremely short cycle; the user works with the system continuously.
- **Weakness:** narrow fit; time-boxing pushes testing and documentation out.

Memory trick: The three R's of RAD: **Rapid**, **R**equirements already known, **R**euse. RAD is what you do once prototyping has made the requirements clear.

> Agile: iterative + incremental + prototyping (Manifesto, **2001**).

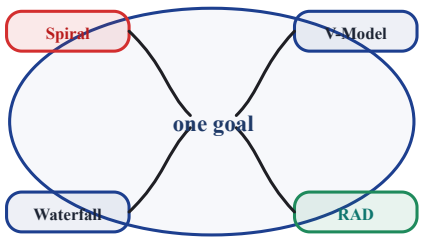
9 Seven Models, One Table: Shape · Fit · Strength · Weakness

Model	Shape	Fits when	Strength	Weakness
Waterfall	one pass, linear	needs known and frozen	documented, budgetable	inflexible; late fix ×100
V-Model	dev left, test right	safety-critical, testable	tests designed early	costly, heavy up front
Prototyping	analyse → prototype → build	needs unclear, UI-heavy	kills ambiguity early	prototype mistaken for product
Incremental	staircase of releases	needs can be prioritised	usable early, risk spread	can cost more than waterfall
Spiral (Boehm)	four-quadrant risk loop	large, high-risk projects	explicit risk analysis	needs risk analysts
RAD	fused analyse+design+build	small team, GUI app	very short cycle	narrow fit; quality squeezed
Big Bang	no phases at all	tiny, low-risk, throwaway	zero process overhead	no feedback, huge late risk

Memory trick: pick the model: ① **Are the requirements clear?** no → **prototyping**; yes → ② **Is the risk high?** yes → **spiral** or **V-model**; no → ③ **Small GUI build?** yes → **RAD**; else **incremental**.

Exam trap: "Early working system?" → incremental / agile / RAD. "A test level planned with each design phase?" → V-model. "Risk-driven?" → spiral. Never mix **iterative** with **incremental**.

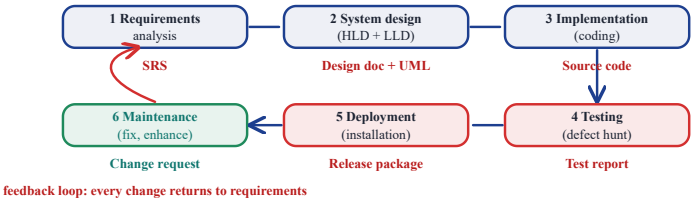
Cross-reference: COCOMO estimation, Gantt / PERT / CPM scheduling and the full risk cycle belong to the **Project Management, Metrics, Risk & Scheduling** lesson.



every model is the same activities in a different shape — the right one, at the right time

10 The SDLC Phases: Activity and Deliverable of Each

SDLC: **S**oftware **D**evelopment **L**ife **C**ycle — the ordered phases from feasibility study to retirement. A written answer must give, for each phase, the **main activity** and the **artefact it produces**.



the flow with its deliverables (red) and the feedback loop back to requirements

Phase	Main activities	Deliverable (artefact)
0 Feasibility	TELOS check — technically, economically, legally, operationally possible?	**Feasibility report**, project charter
1 Requirements analysis & specification	elicit (interviews, questionnaires, observation, prototyping), analyse, specify, validate, manage change	**SRS**, use cases, user & system requirements
2 System design	architecture, modules, interfaces, data design; HLD then LLD; UML diagrams	design document (HLD/LLD), UML class & sequence diagrams
3 Implementation / coding	write the code, unit-test as written, code review, integrate modules	source code, unit-test results, build
4 Testing	verification and validation; unit → integration → system → acceptance	test plan & cases, defect log, report
5 Deployment	install, data migration, user training, transition, post-deployment review	release package, installation & user manual
6 Maintenance	corrective (bug), adaptive (new platform), perfective (feature), preventive (refactoring)	change request, updated release, report

Memory trick: maintenance: **C**orrective = bug · **A**daptive = new OS · **P**erfective = new feature · **P**reventive = stop future bugs (**CAPP**). About **60–80% of the life-cycle cost** is spent here.

Memory trick: cost of a late fix: The later a defect is caught, the dearer the repair: **×1 → ×5 → ×10 → ×15–50 → ×100**.

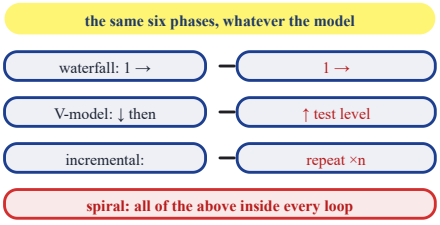
11 Where Does Each Model Place the Phases?

The idea: No model deletes a phase — it **re-orders, repeats, overlaps or fuses** them. That one sentence answers most "compare the models" written questions.

Model	Phases repeated?	When testing starts	Feedback comes from	Phase emphasised
Waterfall	no, one pass	after coding	next phase, late	documentation
V-Model	no, 4 test levels	**planned** with each design	the paired test level	verification
Prototyping	requirements twice	on the prototype	user reaction to a model	requirements
Incremental	yes — **whole SDLC per increment**	in every increment	each release	delivery
Spiral	yes — every loop	in every loop	evaluation per loop	risk analysis
RAD	fused into one box	continuously, 90 days	constant, informal	speed, reuse
Big Bang	none exist	after all is built	the single release	none

Memory trick: Each model answers one question. **Waterfall:** "is it finished?" · **V-model:** "is it tested?" · **Prototyping:** "is it the right thing?" · **Incremental:** "is some of it shipped?" · **Spiral:** "is it safe?" · **RAD:** "is it fast?"

Exam trap: "In which model does **testing begin first**?" — if the option says "at the design of the unit", the answer is the **V-model**. "Which model has **no** phases?" → **Big Bang**; "which repeats the **entire** life cycle?" → **incremental / iterative / spiral**, never waterfall.



the phases never change — only their arrangement does

> Iterative = repeat the process; incremental = add a slice.

Night-Before-the-Exam Sheet

Fact sheet:

SE = systematic, disciplined, quantifiable	Coined at **NATO, Garmisch, 1968**
Attributes: complexity, conformity, changeability, invisibility	Goals: good product · repeatable process · on time, in budget
Qualities: correctness, reliability, efficiency, maintainability, portability, usability	+ integrity, flexibility, testability, reusability
Framework: planning, tracking, risk, QA, config mgmt, reviews	ISO/IEC 12207 · CMM/CMMI 5 levels
Patterns: 5 technology + 3 process + 6 practice = **14**	Waterfall: Benington 1956 · Royce 1970
V-model: req→acceptance, sys→system, arch→integration, module→unit	Spiral: Boehm 1986 · 4 quadrants · risk-driven
RAD: ~90 days, small team, GUI, needs known	Big Bang: no phases at all
SDLC: feasibility, req, design, code, test, deploy, maintain	SRS = requirements-phase deliverable
Maintenance: corrective, adaptive, perfective, preventive	Cost of a fix: ×1 → ×5 → ×10 → ×15–50 → ×100
Info: quant/qual, real-time/hist., internal/external, strat/tact/oper.	Good info: accurate, relevant, complete, timely, reliable

Top 12 one-liners for MCQs

1. **SE** coined at a 1968 NATO conference
2. Waterfall = **linear**, one pass
3. V-model pairs each phase with a **test level**
4. Spiral = **risk-driven** (Boehm, 1986)
5. Big Bang = **no defined phases**
6. SRS is written in **requirements**
7. Prototyping suits **unclear** needs
8. Incremental = **working slices**
9. RAD needs requirements **known**
10. Maintenance is the **largest** cost
11. QA = **reviews + standards**
12. Agile Manifesto = **2001**

Top traps: Royce did not invent the waterfall as a success formula · prototyping ≠ writing the code twice · iterative ≠ incremental · testing is not "only after coding" (V-model).

All tricks together: C-C-C-I for software attributes · CAPP for maintenance kinds · V-model read bottom-up · spiral clockwise: **O**bjectives, **R**isk, **b**uild, **P**lan (ORBP) · three questions (clear? risky? GUI?) to pick a model.



the six phases never change — only the model that arranges them

✓ Practice MCQs (exam pattern, 10 marks)

1. Which model loops plan→risk→build→review?
(a) Waterfall model (b) V-model
(c) Spiral model (d) Big Bang model

2. The SRS document is prepared in which phase?
(a) Analysis & spec. (b) Design
(c) Coding (d) Maintenance

3. Which model pairs each dev phase with a test?
(a) Incremental (b) V-model
(c) Prototyping (d) RAD model

4. Prototyping is most appropriate when:
(a) needs known, fixed (b) small known change
(c) no customer contact (d) needs are unclear

5. "Software Engineering" was coined in 1968 at:
(a) Bangalore (b) Garmisch (NATO)
(c) Cambridge (d) Tokyo

6. Which model has no formal process at all?
(a) Waterfall (b) V-model
(c) Incremental (d) Big Bang

7. The Spiral model was proposed by:
(a) B. W. Boehm (b) J. Martin
(c) K. Beck (d) W. Royce

8. RAD is most suitable for:
(a) air-traffic ctrl (b) small GUI app
(c) a new compiler (d) an OS kernel

9. Maintenance is what share of life-cycle cost?
(a) 5% (b) 20%
(c) 70% (d) 100%

10. A production fix costs how many × more?
(a) 5 (b) 10
(c) 15 (d) 100

Answer key → 1-c · 2-a · 3-b · 4-d · 5-b · 6-d · 7-a · 8-b · 9-c · 10-d

Written question: An agency must build (i) a payroll system with rules fixed by statute, (ii) a citizen portal with unknown needs, and (iii) a payment gateway where a security failure is costly. Name and justify the model for each, with its shape, one strength, one weakness and the phase it emphasises. 10 marks

1. **(i) Payroll → Waterfall + V-model tests.** Linear, signed off; budgetable, but a statute change forces rework. 4
2. **(ii) Portal → Prototyping, delivered incrementally.** Kills ambiguity first. 3
3. **(iii) Gateway → Spiral (risk-driven).** Risks retired early; expensive. 3

Before the exam ✓

- ✓ Can give the IEEE definition and 1968 origin
- ✓ Can list 7 models and draw each shape
- ✓ Can recite V-model pairings from module → unit

Also practise (written): ① Name the deliverable of each of the 6 phases. ② Draw the spiral model with its four quadrants. ③ Iterative vs incremental development.